National Institute of Electronics and Information Technology, Chennai

Online Internship in RTL Design using Verilog HDL

Project Report

On

Low-Power Wallace Tree Multiplier

Submitted By

Shaikh Jaheed

at

18 February 2026

About NIELIT

National Institute of Electronics and Information Technology (NIELIT), headquartered in New Delhi is the capacity building arm of the Ministry of Electronics and Information Technology (MeitY), mandated to carry out HR development and related activities in Information, Electronics and Communication Technology (IECT).

Since its inception in 2010, NIELIT Chennai has established itself as a premier institution providing affordable quality education as per the job market requirements for candidates primarily from Tamil Nadu. As of date NIELIT Chennai has imparted skill training to more than 30, 000 youth of this region and conducted digital literacy certification for more than 25,000 candidates. NIELIT Chennai handles the activities of the NIELIT in Tamil Nadu, Andhra, Telangana, and the Andaman and Nicobar Islands.

Presently NIELIT Chennai is implementing the following projects for Tamil Nadu: ISEA (Information Security Education and Awareness), Future Skills PRIME capacity building projects (3D Printing and Additive Manufacturing, Robotic Process Automation, IoT - technology streams), and the aspirational district projects for SC/ST candidates at Ramanathapuram and Virudhunagar funded by MeitY, Government of India. The centre also received funding through the electronics product design and product testing capacity building project funded by MeitY.

Other than training, the centre also provides services like; Private Cloud Setup using Citrix, vSphere, Red Hat & FOSS Cloud, Hyper-converged Infrastructure (HCI) and Virtual Desktop Infrastructure (VDI) Server Setup, High-Performance Computing (HPC) Setup with 100G/200G connectivity, Server and Virtual Lab Setup for Bigdata, AI & Information Security, and Digital & Mobile Forensics Service.

NIELIT has also set up a Virtual Academy, a common End-to-End Platform to conduct Online training courses including a virtual lab environment in the areas of IECT from foundation to expert level targeting from school students to working professionals.

INDEX

Contents	Page No.
ACKNOWLEDGEMENT	1
ABSTRACT	2
LIST OF FIGURES	3
CHAPTER 1: INTRODUCTION	4
CHAPTER 2: LITERATURE	5
CHAPTER 3: HARDWARE TOOLS AND COMPONENT	6
CHAPTER 4: SOFTWARE TOOLS AND LANGUAGES	7
CHAPTER 5: IMPLEMENTATION	8
CHAPTER 5.1: FLOW CHART	10
CHAPTER 6: RESULTS AND DISCUSSION	11
CHAPTER 7: CONCLUSION AND DISCUSSION	17
CHAPTER 8: REFERENCE	18

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have supported and guided me throughout the completion of this project on **Low-Power Wallace Tree Multiplier** as part of the VL151-2 'O' Level course, conducted by the National Institute of Technology and Applied Research (NIELIT), Chennai.

First and foremost, I would like to thank my course instructor, Ishant Kumar Bajpai (Scientist 'D') , for their invaluable guidance, constructive feedback, and constant encouragement throughout the project. Their expertise in the field of Chip Design has been a key factor in the successful completion of this report.

I would also like to extend my appreciation to the NIELIT Chennai for offering this comprehensive online course, providing us with high-quality learning resources and support despite the challenges of remote learning.

A special thanks to my fellow participants in the online course for the engaging discussions and knowledge-sharing that enriched my learning experience. The collaborative nature of this course has significantly enhanced the depth and quality of my work.

Finally, I would like to thank all the authors, researchers, and resources referenced in this report for their contributions to the field of Chip Design, which formed the foundation of my project.

Thank you to everyone who has been part of this learning journey.

ABSTRACT

This project uses Verilog HDL to design and implement a low-power 8x8 Wallace tree multiplier. Because of its parallel partial product reduction technique, which drastically lowers computation delay when compared to traditional array multipliers, the Wallace Tree architecture is frequently used in high-speed multiplication. The operand isolation technique is employed in this project to achieve a low-power design by lowering dynamic power consumption and switching activity.

The Artix-7 FPGA is the target of the Xilinx Vivado 2025.1 simulation, synthesis, and implementation of the design. Static timing analysis, implementation, power analysis, behavioural simulation, and post-synthesis simulation are all carried out. The outcomes verify proper operation, adherence to timing, and lower power usage.

This project implements an **8x8 Wallace Tree Multiplier** using Verilog HDL, specifically optimized for low-power operation. Unlike standard multipliers that add partial products one by one, the Wallace Tree uses a tree-like structure of Half and Full Adders to reduce partial products in parallel. Speed of Wallace tree multiplier can be improved by using compressor methods. By minimizing the number of partial products, we used half adder and full adder in 8-bit multiplier. In this Wallace tree multiplication process we also used carry save adder in the last stage to accumulate the last bits. In this work 8*8-bit Wallace tree multiplier construction is examined and simulated in XILINX software. In this 8-bit Wallace tree multiplier circuit our main goals are to decrease the area of multiplier circuit and increase the speed of multiplier.

LIST OF FIGURES

S.NO.	TITLE	PAGE NO.
1	RTL Schematic	9
2	Behavioral Simulation	11
3	Synthesis Utilization	12
4	Post-synthesis functional & timing simulation	13
5	Static timing analysis	14
7	Post-Implementation Simulation	15
8	Power report	16

CHAPTER 1
INTRODUCTION

Multiplier plays an important role in today's digital signal processing and diversity of other applications. There are many multipliers are used in the digital signal processing system like binary multiplier, tree multiplier, Array multiplier etc. With advances in technology, a lot of researchers have tried and are trying to design multipliers which recommend any of the following design targets – high speed, low power utilization, regularity of layout and hence minimum area or even combination of them in one multiplier therefore making them suitable for a variety of high speed, low power and dense VLSI implementation.

The common multiplication method is "add and shift" algorithm. In parallel multipliers number of partial products to be added is the main parameter that determines the performance of the multiplier. To reduce the number of partial products to be added, Modified Booth algorithm is one of the most popular algorithms. To achieve speed improvements Wallace Tree algorithm can be used to reduce the number of sequential adding stages. Further by combining both Modified Booth algorithm and Wallace Tree technique we can see advantage of both algorithms in one multiplier. However, with increasing parallelism, the number of shifts between the partial products and intermediate sums to be added will increase which may result in reduced speed, increase in silicon area due to irregularity of structure and also increased power consumption due to increase in interconnect resulting from complex routing. On the other hand, "serial-parallel" multipliers compromise speed to achieve better performance for area and power consumption.

CHAPTER 2
LITERATURE

Conventional array multipliers suffer from high delay due to sequential addition of partial products. Wallace Tree architecture reduces delay by grouping partial products and reducing them in parallel stages using full adders and half adders.

Various low-power techniques such as clock gating, operand isolation, and voltage scaling are used in modern VLSI design to reduce power consumption. In this project, operand isolation is implemented to reduce unnecessary switching activity when the multiplier is not in use.

Statics shows that more than 70% instructions in microprocessor and mainly DSP algorithms perform addition and multiplication. High speed is main issue to design a multiplier for a designer. The speed of a multiplier determines its performance. High speed multiplier is an essential requirement for all multiplication methods. Low power utilization is also a main topic in multiplier design. To decrease major power utilization, it is good to decrease the number of operations by this means decreasing dynamic power which is a main part of total power utilization so the need of high speed and low power multiplier has increased. Designer mainly focuses on high speed and low power efficient circuit design. The purpose of a good multiplier is to provide a physically packed together, high speed and low power utilization unit.

A Wallace tree is a very useful hardware function that is used in digital circuit for multiplies two integers. In Wallace method the multiplication of two numbers is done by reducing the partial product matrix into a two-row matrix by a half adder, full adder, carry-save adder & these two rows are added utilizing a fast carry propagate adder to produce the output product. In this Wallace tree method, we used half adder for summation of 2 bit and used full adder for summation of 3 bit. For multiplicands of higher than 8-bits this advantage is more beneficial, because the addition of partial products is low in Wallace tree and hence increases speed. Here each bit of each partial product in every column is added together by a set of counters used in parallel so that no carry is propagated further. Then this matrix is reduced by another set of counters until a two-row matrix generates.

CHAPTER 3
HARDWARE TOOLS AND COMPONENT

- FPGA Device: Xilinx Artix-7 (xc7a200t-fbg676-2)
- Target Board: Basys-3 (if applicable)
- Computer System: Windows 32-bit System

CHAPTER 4
SOFTWARE TOOLS AND LANGUAGES

- Software Tool: Xilinx Vivado 2025.1
- Language Used: Verilog HDL
- Simulation Tool: Vivado Simulator
- Design Type: RTL Design

CHAPTER 5
IMPLEMENTATION

The implementation consists of the following stages:

1. **Writing Verilog modules:**
    - Half Adder
    - Full Adder
    - 8×8 Wallace Tree Multiplier
    - Testbench
2. **Behavioral Simulation:**
   Functional verification of multiplier output.
3. **Synthesis:**
   Conversion of RTL code into gate-level netlist.
4. **Static Timing Analysis:**
   Verification of timing constraints and slack.
5. **Implementation:**
   Placement and routing of the design on FPGA fabric.
6. **Power Analysis:**
   Estimation of total on-chip power consumption.

Operand isolation is implemented using an enable signal to reduce switching activity when the multiplier is not active.

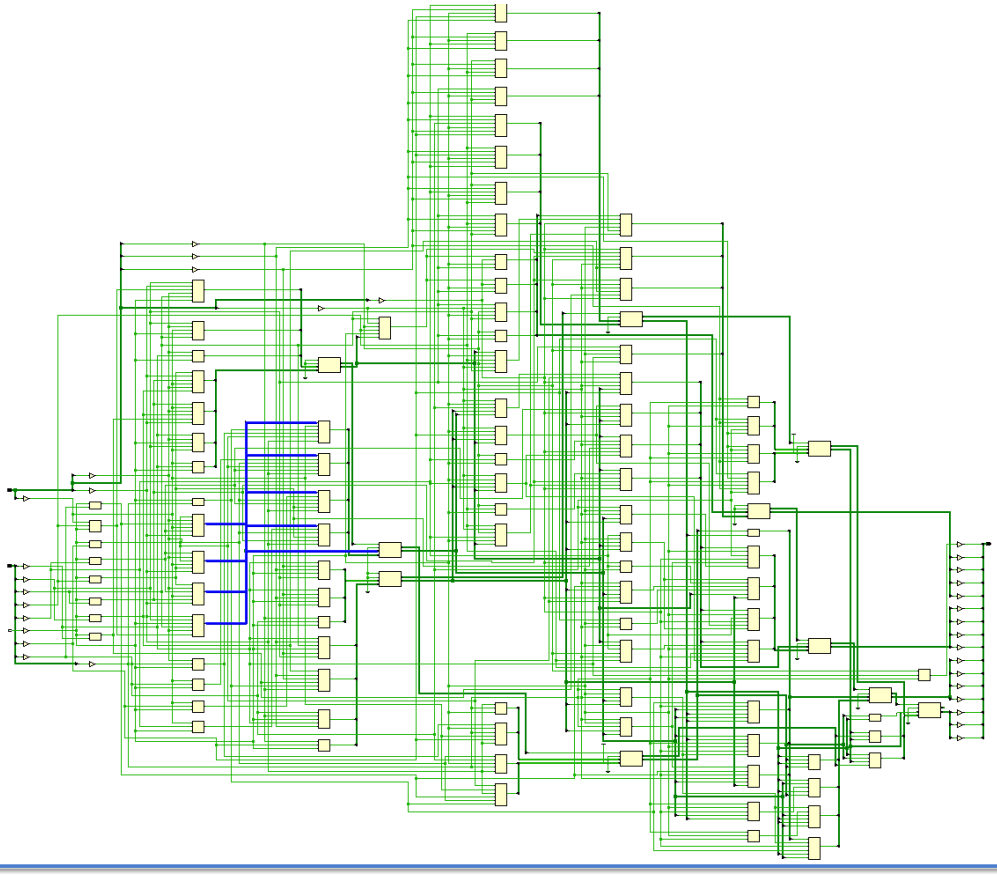
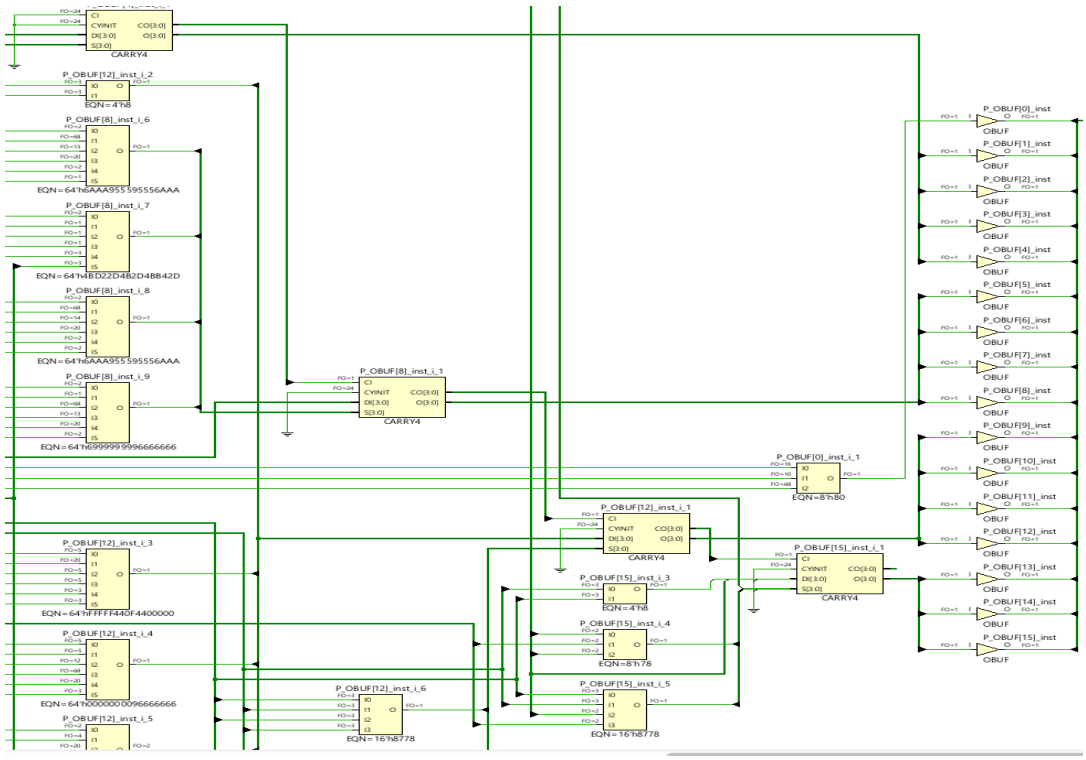
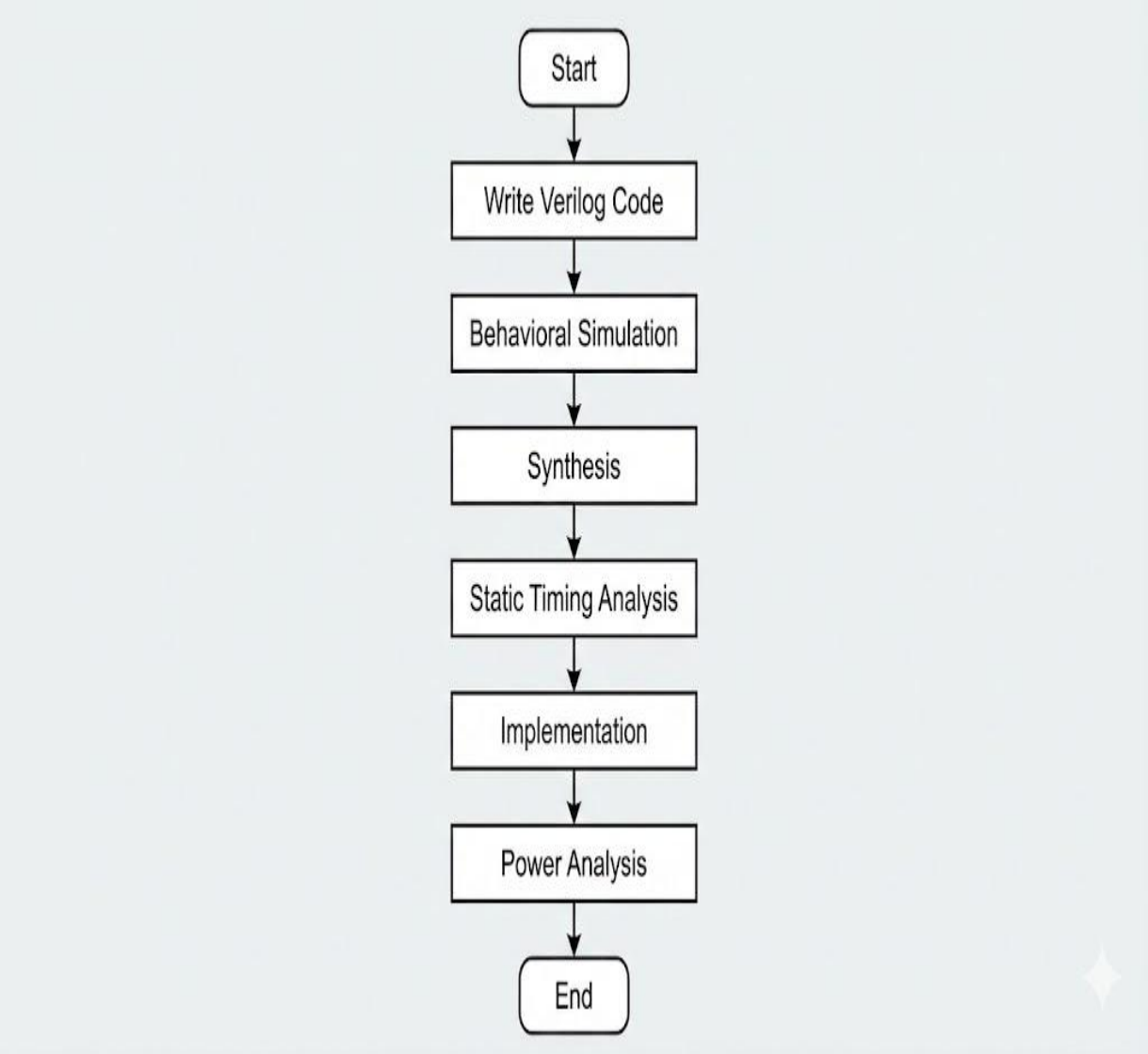


Figure 1: RTL Schematic of 8×8 Low-Power Wallace Tree Multiplier



CHAPTER 5.1
FLOW CHART



CHAPTER 6
RESULTS AND DISCUSSION

The behavioral simulation confirmed correct multiplication results for various input combinations. Post-synthesis and post-implementation simulations verified functional correctness. The timing summary showed no timing violations. Since the design is combinational, infinite slack was reported due to absence of clock constraints. The power analysis report indicated reduced dynamic power due to operand isolation technique.

The Wallace Tree architecture achieved lower delay compared to conventional array multiplier architecture. In this multiplication process first, we implement the programme in Xilinx Software then we see the RTL view of this 8-bit Wallace tree multiplication process. In this RTL view we see the interconnection of half adder and full adder with Sum and Carry signal.

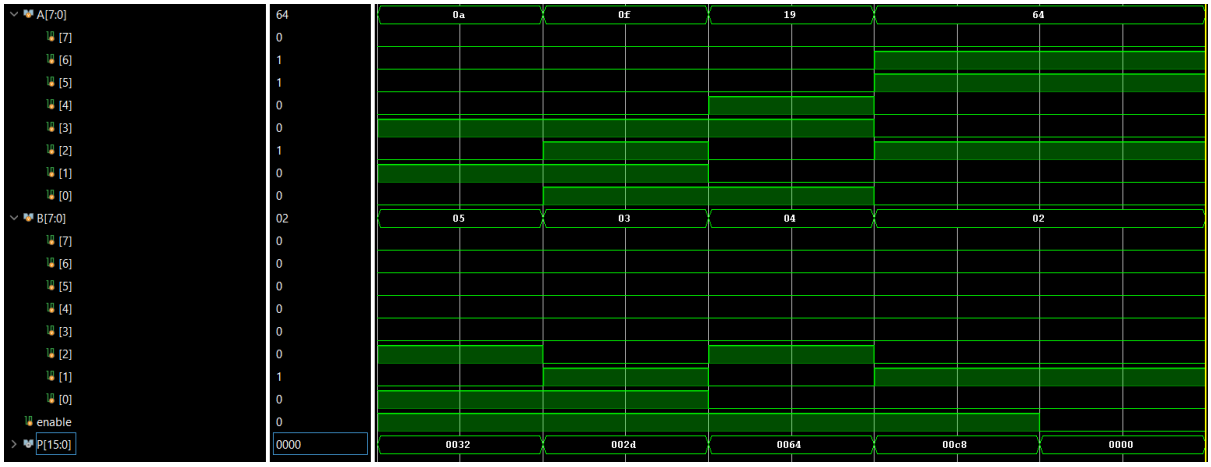
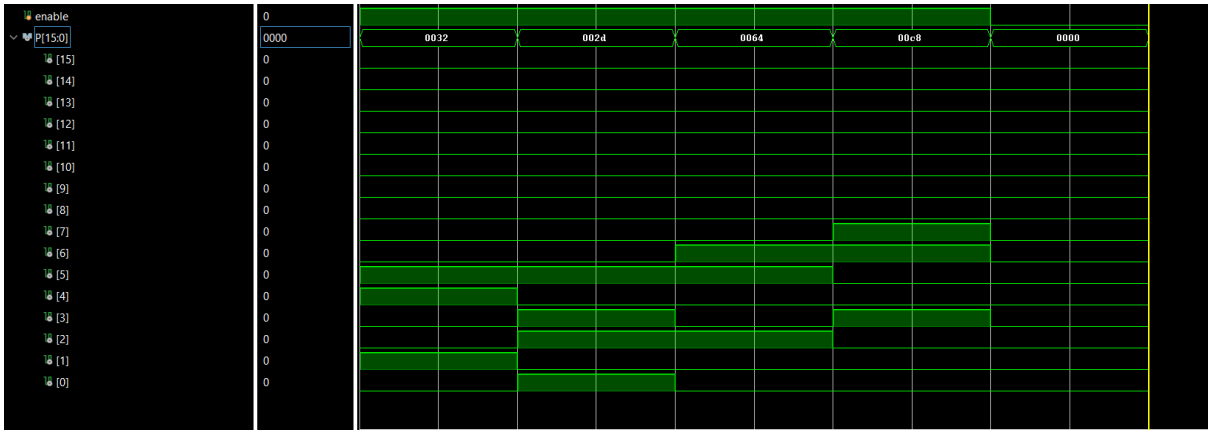


Figure 2: Behavioral Simulation Waveform



3. DSP

Site Type	Used	Fixed	Prohibited	Available	Util%
DSPs	0	0	0	740	0.00

4. IO and GT Specific

Site Type	Used	Fixed	Prohibited	Available	Util%
Bonded IOB	33	0	0	400	8.25
Bonded IPADs	0	0	0	26	0.00
Bonded OPADs	0	0	0	16	0.00
PHY_CONTROL	0	0	0	10	0.00
PHASER_REF	0	0	0	10	0.00
OUT_FIFO	0	0	0	40	0.00
IN_FIFO	0	0	0	40	0.00
IDELAYCTRL	0	0	0	10	0.00
IBUFDS	0	0	0	384	0.00
GTPE2_CHANNEL	0	0	0	8	0.00
PHASER_OUT/PHASER_OUT_PHY	0	0	0	40	0.00
PHASER_IN/PHASER_IN_PHY	0	0	0	40	0.00
IDELAYE2/IDELAYE2_FINEDELAY	0	0	0	500	0.00
IBUFDS_GTE2	0	0	0	4	0.00
ILOGIC	0	0	0	400	0.00
OLOGIC	0	0	0	400	0.00

7. Primitives

Ref Name	Used	Functional Category
LUT6	41	LUT
LUT5	23	LUT
LUT3	19	LUT
IBUF	17	IO
OBUF	16	IO
LUT2	10	LUT
CARRY4	10	CarryLogic
LUT4	4	LUT

Figure 3: Synthesis Resource Utilization

Figure 4.1: Post-Synthesis Functional Simulation

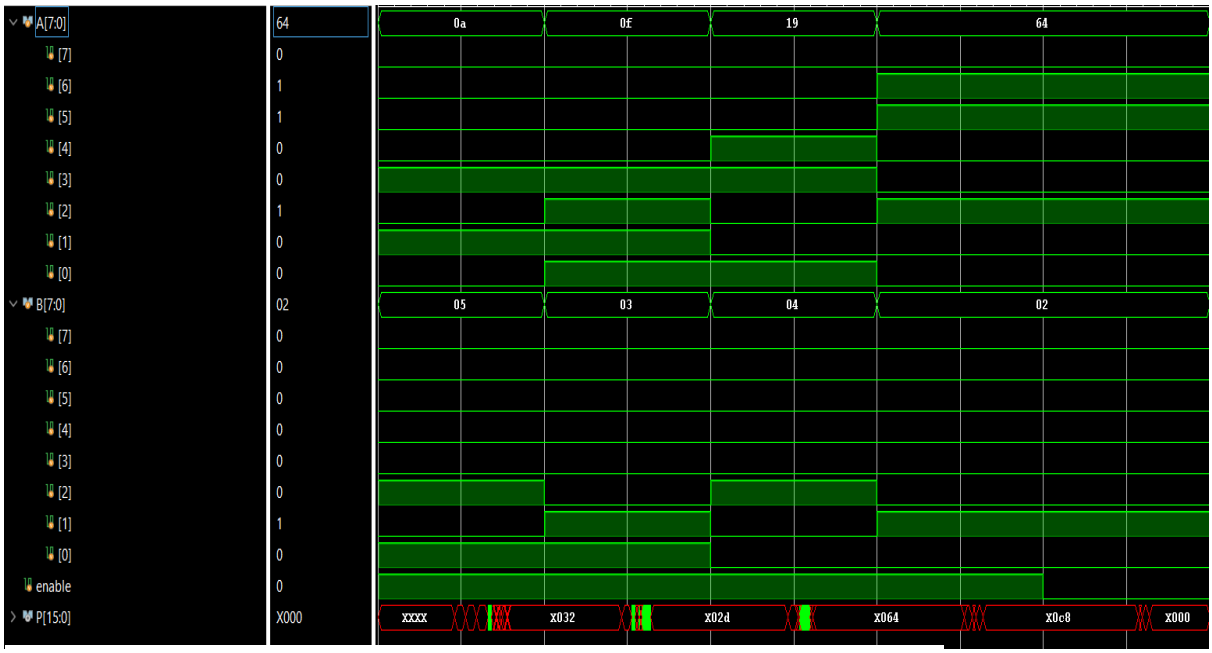


Figure 4.2: Post-Synthesis Timing Simulation

```
Max Delay Paths
--------------------------------------------------------------------------------------
Slack:                    inf
  Source:                 enable
                            (input port)
  Destination:            P[15]
                            (output port)
  Path Group:             (none)
  Path Type:              Max at Slow Process Corner
  Data Path Delay:        11.879ns  (logic 5.306ns (44.665%)  route 6.573ns (55.335%))
  Logic Levels:           11  (CARRY4=4 IBUF=1 LUT2=1 LUT5=1 LUT6=3 OBUF=1)

    Location             Delay type                Incr(ns)  Path(ns)    Netlist Resource(s)
  -------------------------------------------------------------------    -------------------
    P23                                                 0.000     0.000 f  enable (IN)
                         net (fo=0)                     0.000     0.000    enable
    P23                  IBUF (Prop_ibuf_I_O)           0.896     0.896 f  enable_IBUF_inst/O
                         net (fo=68, routed)            2.050     2.945    enable_IBUF
    SLICE_X2Y111         LUT2 (Prop_lut2_I0_O)          0.105     3.050 r  P_OBUF[4]_inst_i_12/O
                         net (fo=5, routed)             0.930     3.980    P_OBUF[4]_inst_i_12_n_0
    SLICE_X2Y113         LUT6 (Prop_lut6_I3_O)          0.105     4.085 r  P_OBUF[8]_inst_i_31/O
                         net (fo=1, routed)             0.000     4.085    P_OBUF[8]_inst_i_31_n_0
    SLICE_X2Y113         CARRY4 (Prop_carry4_S[2]_CO[3])
                                                        0.316     4.401 r  P_OBUF[8]_inst_i_12/CO[3]
                         net (fo=1, routed)             0.000     4.401    P_OBUF[8]_inst_i_12_n_0
    SLICE_X2Y114         CARRY4 (Prop_carry4_CI_O[1])
                                                        0.257     4.658 r  P_OBUF[12]_inst_i_11/O[1]
                         net (fo=2, routed)             0.317     4.975    P_OBUF[12]_inst_i_11_n_6
    SLICE_X3Y114         LUT5 (Prop_lut5_I4_O)          0.245     5.220 f  P_OBUF[12]_inst_i_13/O
                         net (fo=2, routed)             0.776     5.996    P_OBUF[12]_inst_i_13_n_0
    SLICE_X0Y114         LUT6 (Prop_lut6_I0_O)          0.105     6.101 r  P_OBUF[12]_inst_i_5/O
                         net (fo=2, routed)             0.573     6.675    P_OBUF[12]_inst_i_5_n_0
    SLICE_X1Y114         LUT6 (Prop_lut6_I0_O)          0.105     6.780 r  P_OBUF[12]_inst_i_9/O
                         net (fo=1, routed)             0.000     6.780    P_OBUF[12]_inst_i_9_n_0
    SLICE_X1Y114         CARRY4 (Prop_carry4_S[0]_CO[3])
                                                        0.440     7.220 r  P_OBUF[12]_inst_i_1/CO[3]
                         net (fo=1, routed)             0.000     7.220    P_OBUF[12]_inst_i_1_n_0
    SLICE_X1Y115         CARRY4 (Prop_carry4_CI_O[2])
                                                        0.200     7.420 r  P_OBUF[15]_inst_i_1/O[2]
```

```
--------------------------------------------------------------------------------------
| Design Timing Summary
| ---------------------
--------------------------------------------------------------------------------------

    WNS(ns)      TNS(ns)  TNS Failing Endpoints  TNS Total Endpoints      WHS(ns)      THS(ns)  THS Failing Endpoints  THS Total Endpoints     WPWS(ns)     TPWS(ns)  TPWS Failing Endpoints
    -------      -------  ---------------------  -------------------      -------      -------  ---------------------  -------------------     --------     --------  ----------------------
        inf        0.000                      0                   16          inf        0.000                      0                   16           NA           NA                      NA
```

Figure 5: Static Timing Analysis Report

The timing report confirms that there are no timing violations.

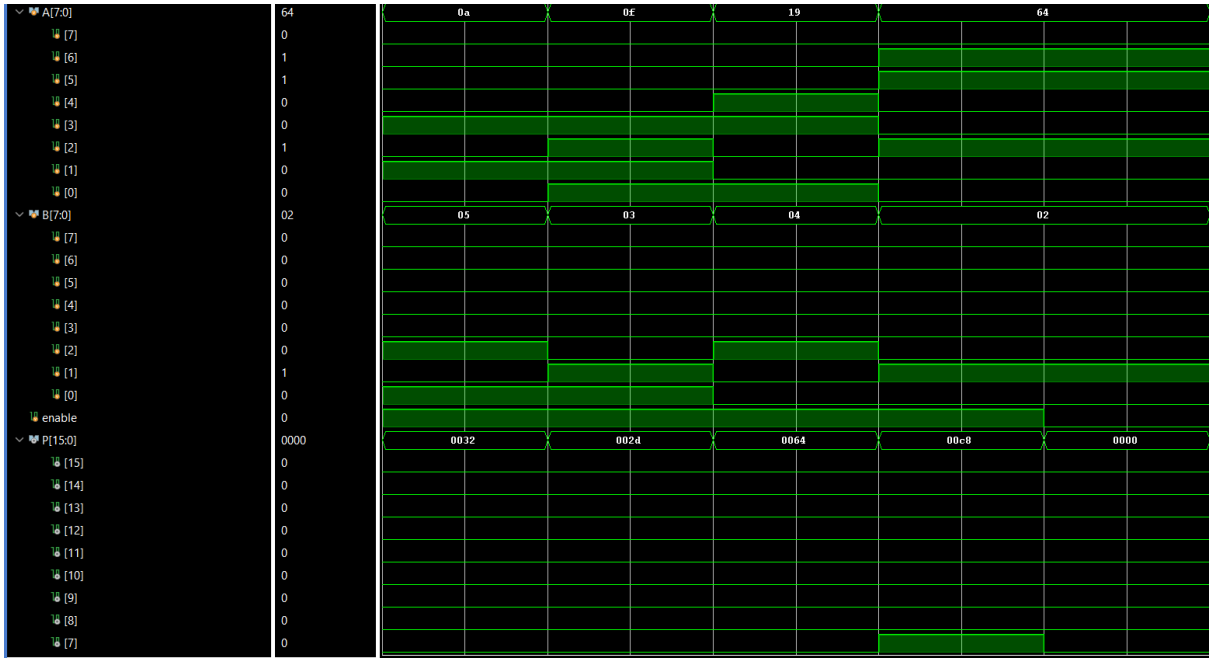


Figure 6.1: Post-Implementation Functional Simulation

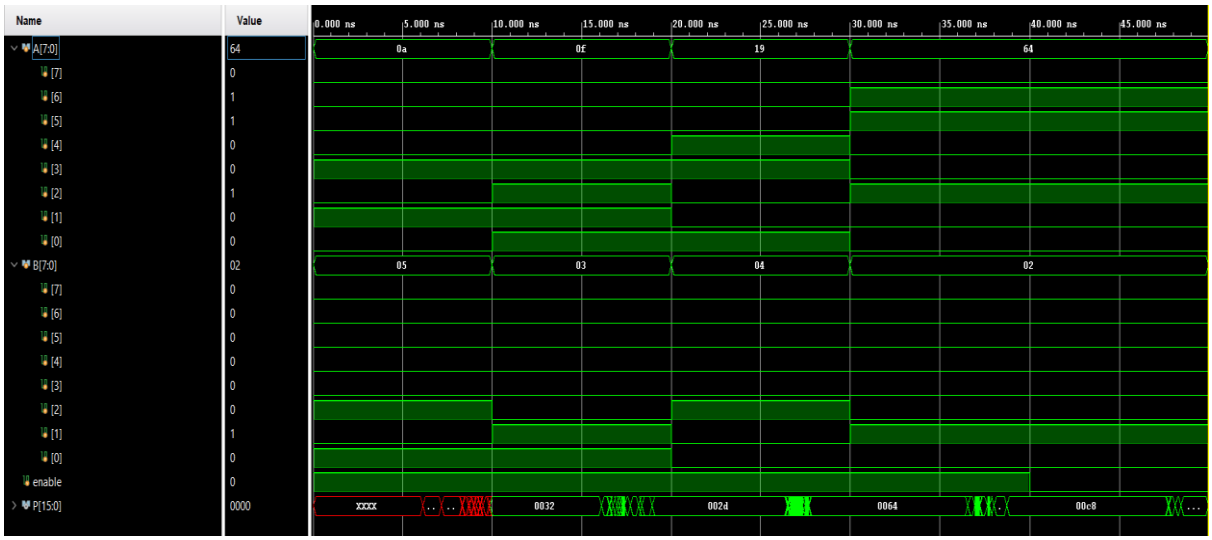


Figure 6.2: Post-Implementation Timing Simulation

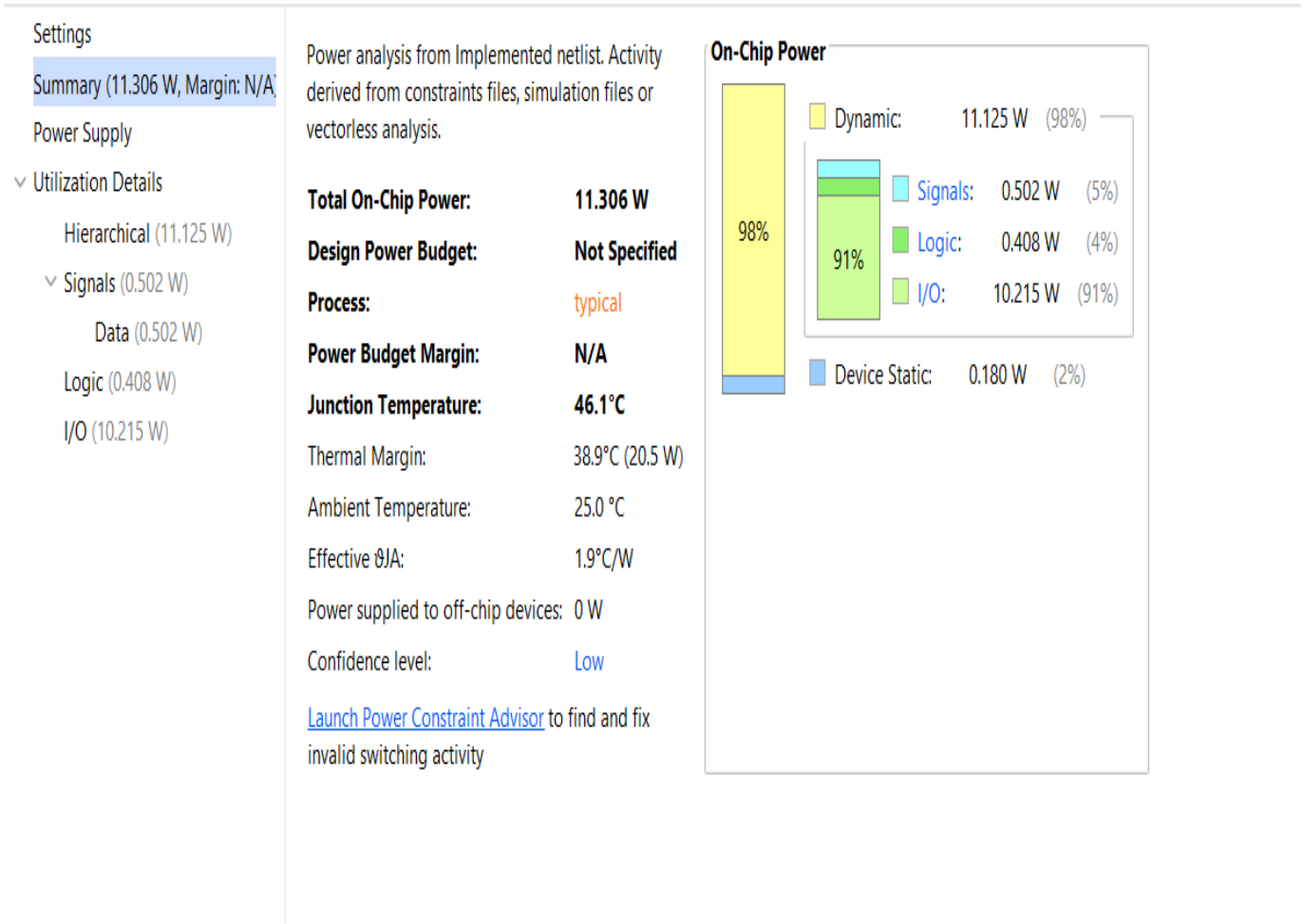


Figure 7: Power report

CHAPTER 7
CONCLUSION AND DISCUSSION

This report offers the multiplication process of 8 bit using Wallace tree. In this work we used VHDL structural type programming language to multiply the 8 unsigned bits. In this 8-bit Wallace tree multiplication process, we used half adder, full adder & carry save adder to reduce the bit in the multiplication process. In this Wallace tree multiplication process, we used five stages to reduce all the bits. We are going to use Xilinx 14.6 Ise design suite software for implementation and simulation of this multiplication process. In the implementation of Wallace tree multiplication process, we got an output image of RTL view. In the simulation process, we got an output graphical images of all input, output and signal value.

The Low-Power 8×8 Wallace Tree Multiplier was successfully designed and implemented using Verilog HDL. The design was simulated, synthesized, and implemented in Vivado. Operand isolation effectively reduced switching activity and dynamic power consumption.

The project demonstrates how parallel reduction techniques improve speed while low-power techniques enhance efficiency in digital systems.

CHAPTER 8
REFERENCE

1. **Kai Hwang "Computer Arithmetic: Principles, Architecture, and Design" John Wiley & Sons 1979**
2. **S. D. Pezaris "A 40-ns 17-Bit by 17-Bit Array Multiplier", IEEE Trans. on Computers, pp. 442-447,.Apr. 1971**
3. **Neeta Sharma, Ravi Sindal "Modified Booth Multiplier using Wallace Structure and Efficient Carry Select Adder",International Journal of Computer Applications (0975 – 8887) Volume 68– No.13, pp 40-42,April 2013.**
4. **Weste, Neil H.E. Eshraghian, and kamran," CMOS VLSI Design: A Circuit and System Prespective", 3rd Edition, Pearson Education, pp. 345-356, 2005.**
5. **Xilinx Vivado Design Suite User Guide**